



백엔드 ToDo

☰ 팀	
⚙ 상태	시작 전

Capstone_Back — TODO 목록

코드 전체 분석을 바탕으로 미구현/미완성 항목을 정리했습니다.

● 필수 → ● 중요 → ● 개선 순으로 우선순위를 표시했습니다.

● 필수 — 핵심 기능 미구현

1. `/api/evaluate` 엔드포인트 구현 (`routers/evaluate.py`)

현재 상태:

```
# routers/evaluate.py
router = APIRouter(prefix="/api", tags=["Evaluation"])
# TODO: 앞으로 여기에 [POST] /evaluate 엔드포인트를 추가할 예정입니다.
```

해야 할 일:

- ☐ [] `POST /api/evaluate` 엔드포인트 작성
- ☐ [] 요청 바디 스키마(`EvaluateRequest`) 정의 — `schemas.py` 에 추가 필요
 - 파일(CSV/JSON) or 이전에 분석한 데이터 재사용 방식 결정
 - `confirmed_mappings` , `task_type` , `selected_tcs` , `metadata` 포함
- ☐ [] 파일 업로드 → 파싱 → `evaluator/engine.evaluate()` 호출 흐름 연결
- ☐ [] 평가 결과를 응답 바디 스키마(`EvaluateResponse`)로 감싸서 반환

2. `report.py` 구현 (`evaluator/report.py`)

현재 상태:

```
def generate_report(results: Dict[str, Any]) -> Dict[str, Any]:  
    """계산된 지표 결과들을 최종 API 응답 형태로 포맷팅합니다."""  
    pass # 미구현
```

해야 할 일:

- ☐ [] 계산된 TC 결과들을 보기 좋은 구조로 포맷팅
- ☐ [] `_metadata` (전처리 로그: 제거된 행 수, 경고 목록)를 결과에 포함
- ☐ [] 오류가 발생한 TC는 별도로 분리해서 반환
- ☐ [] (선택) ISO/IEC TS 4213:2022 기준 리포트 형식 고려

● 중요 — 스키마 / 구조 보완

3. `schemas.py` 에 평가 요청/응답 스키마 추가

현재 `schemas.py` 에는 분석(Step 1, 2) 스키마만 있고, 평가(Step 3) 스키마가 없습니다.

- ☐ [] `EvaluateRequest` 모델 추가

```
class EvaluateRequest(BaseModel):  
    task_type: TaskType  
    column_mappings: list[ColumnMapping]  
    selected_tcs: list[str]  
    metadata: DataMetadata # positive_class 등 평가에 필  
    요한 메타 정보
```

- ☐ [] `EvaluateResponse` 모델 추가

```
class EvaluateResponse(BaseModel):  
    results: dict[str, Any] # TC별 계산 결과  
    warnings: list[str] # 전처리 경고 목록  
    dropped_rows: int # 제거된 행 수
```

4. `engine.py` 의 역할 키 변환 정리

현재 `engine.py` 에서 `y_true → true_class` 같은 변환을 즉석에서 처리하고 있는데,

이는 `schemas.py` 와 `metrics/` 사이의 불일치에서 비롯된 임시 코드입니다.

☐ [] `metrics` 함수들의 내부 키 이름을 `y_true` , `y_pred` 로 통일하거나
OR 변환 로직을 별도 유틸 함수로 분리해서 관리 포인트를 줄임

☐ [] `VALID_TCS_BY_TASK` 와 `validator.py` 의 `_TC_REQUIREMENTS` 가 중복 관리되고 있음 →
단일 소스로 통합 고려

5. `positive_class` 정보를 평가에 활용

현재 `analyzer.py` 에서 Binary의 양성 클래스를 자동으로 판별해 `DataMetadata` 에 저장하지만,
만,

실제 평가(`binary.py`)에서는 이 정보를 사용하지 않고 **정렬 기반으로 임의 판단**합니다.

```
# binary.py - 현재
pos_label = classes[-1] # 정렬 후 마지막 값을 Positive로 간주
(임의!)
```

☐ [] `EvaluateRequest` 에 `metadata.positive_class` 를 포함시켜 `calculate_auroc` ,
`calculateauprc` , `calculate_log_loss` 에 전달

☐ [] `_binarize_true_labels` 가 `positive_class` 를 인자로 받을 수 있도록 수정

● 개선 — 코드 품질 / 테스트

6. 테스트 보완

현재 `test_analyze.py` , `test_evaluator.py` 두 파일이 있지만 분석 필요.

☐ [] `test_evaluator.py` — Binary / Multiclass / Multilabel 각 TC 계산 결과 검증

☐ [] 전처리 예외 케이스 테스트 (NaN 전체, 확률 범위 초과, 컬럼 누락 등)

☐ [] `validate_mapping` 엣지 케이스 테스트 (중복 역할, Binary `y_pred·score` 둘 다 없음 등)

7. `requirements.txt` 보완

현재 `numpy` 가 명시되어 있지 않습니다. (pandas 의존성으로 암묵적으로 설치되지만)

☐ [] `numpy` 명시적으로 추가

☐ [] 버전 고정 확인 (`scikit-learn==1.4.2` → numpy 호환 버전 명시)

8. F-beta의 beta 값 전달 경로 미구현

현재 TC5(F-beta Score)는 `beta=1.0`으로 하드코딩되어 있고, 주석으로만 언급됩니다.

```
# common.py
def calculate_fbeta_score(df, mapping_dict, beta: float = 1.0) -> float:
    # 차후 프론트에서 beta 값을 넘겨주면 활용 가능하도록 설계
```

☐ [] `EvaluateRequest`에 `beta` 파라미터 추가

☐ [] `engine.py`에서 TC5 호출 시 beta 값을 전달하도록 수정

전체 현황 요약

항목	파일	상태
<code>/api/evaluate</code> 엔드포인트	<code>routers/evaluate.py</code>	● 미구현
<code>generate_report()</code>	<code>evaluator/report.py</code>	● 미구현 (<code>pass</code>)
평가 요청/응답 스키마	<code>schemas.py</code>	● 없음
<code>positive_class</code> 평가 연동	<code>evaluator/metrics/binary.py</code>	● 임시 코드
역할 키 이름 불일치 정리	<code>engine.py</code>	● 기술 부채
TC 가용성 중복 관리	<code>engine.py</code> + <code>validator.py</code>	● 기술 부채
테스트 보완	<code>test_evaluator.py</code> 등	● 미비
<code>numpy</code> requirements 명시	<code>requirements.txt</code>	● 누락
F-beta beta 값 전달	<code>engine.py</code> + <code>schemas.py</code>	● 미연결